

Why Bad Code Quality Is Not (Only) Your Fault

Annelore Egger



BERLIN EXPERT DAYS 2026

OPENVALUE
Better software, faster

About Me

- BSc in Computer Science
- 8 years of programming experience
- Software Engineer, OpenValue Switzerland since May 2024



THE TUG-OF-WAR ON CODE QUALITY

DEVELOPERS

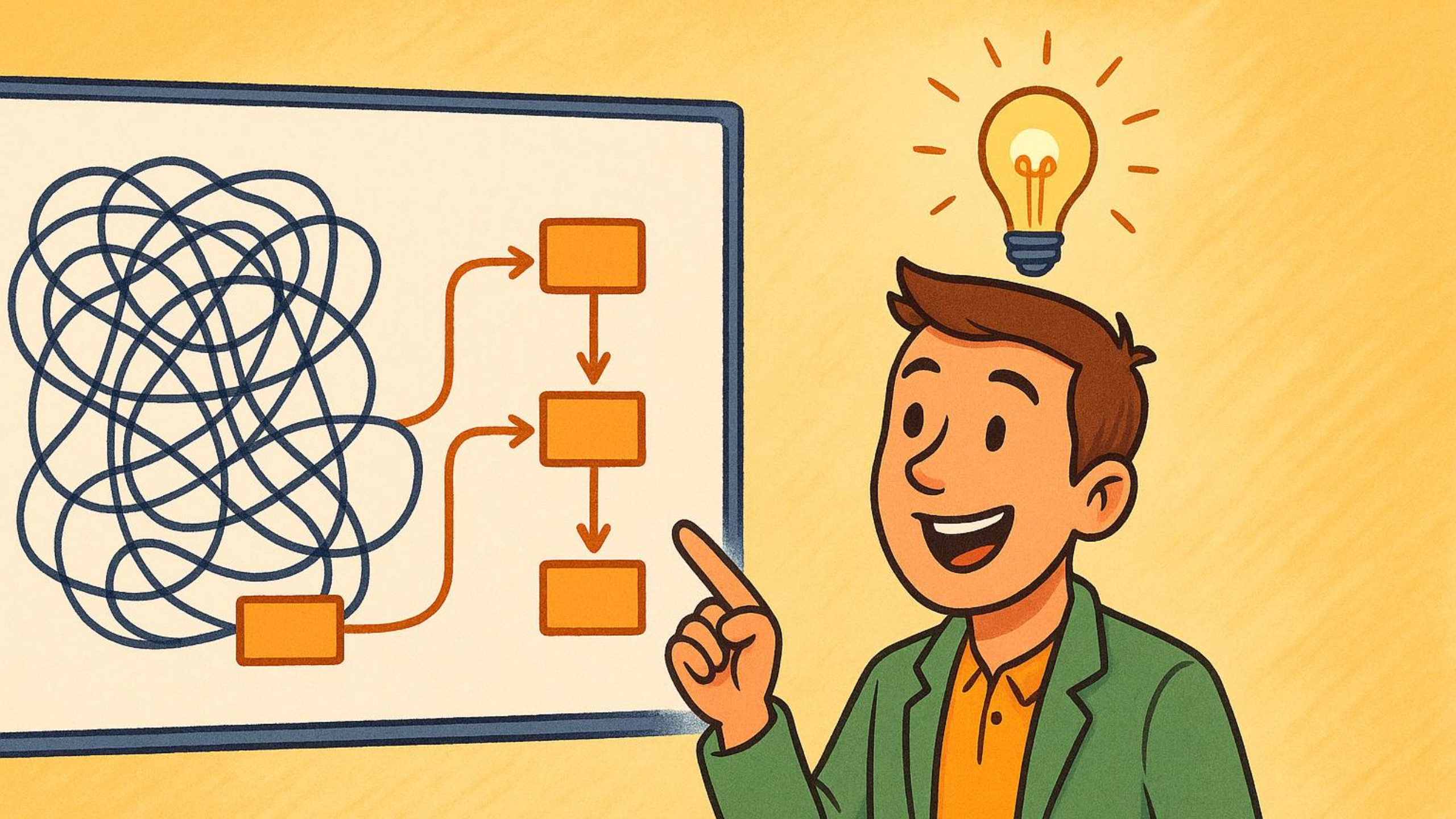


**BUSINESS
MANAGERS**



Code Quality

code that is easy to understand, easy to
change, and less likely to break
unexpectedly



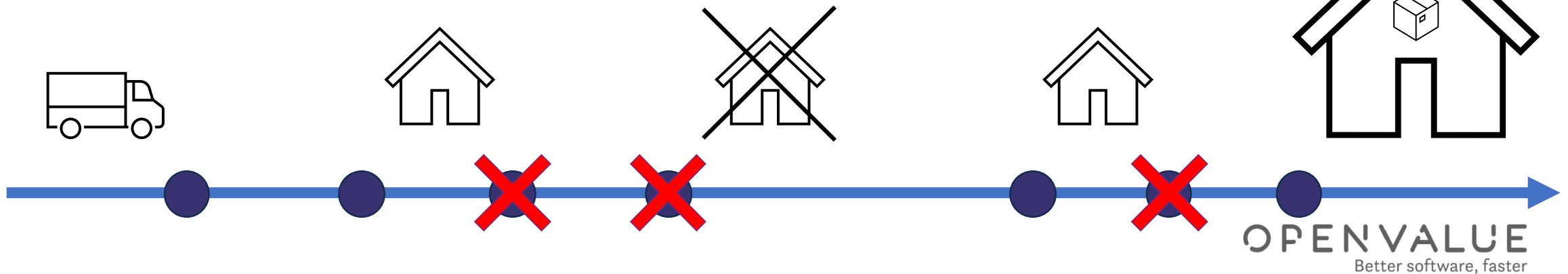
Scenario

Legend

- updates sent to central system
- ✗ updates no longer necessary

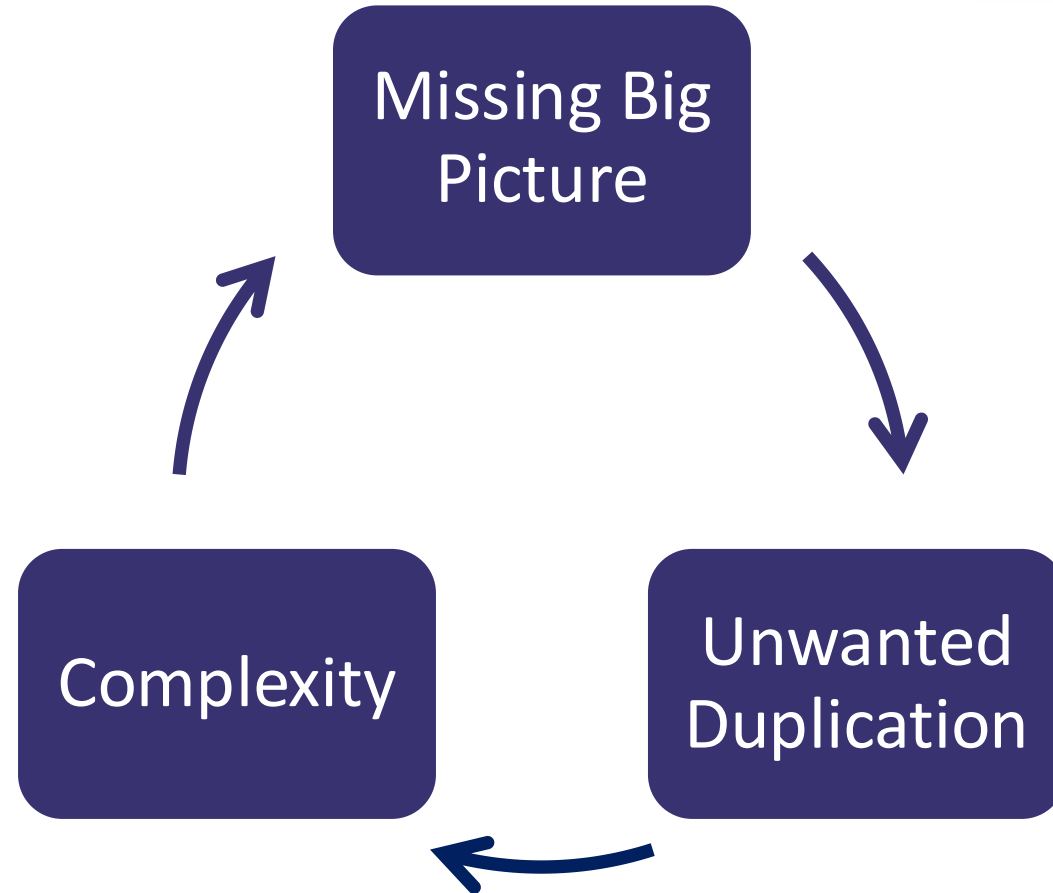


warehouse



Factors That Impact Code Quality (and What We Can Do About Them)

Underlying Issues



Miscommunication and Misunderstandings

Misunderstanding Between Stakeholders and Developers

- Stakeholders and developers do not speak the same language
- Try to get rid of assumptions
 - Ask questions
 - Give background/context-information

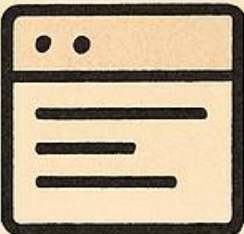
Quote: Stakeholder-Developer Misalignment

“What is deployed to production is your
developer’s misunderstanding”
- Jacqui Read

Mitigating Misunderstandings

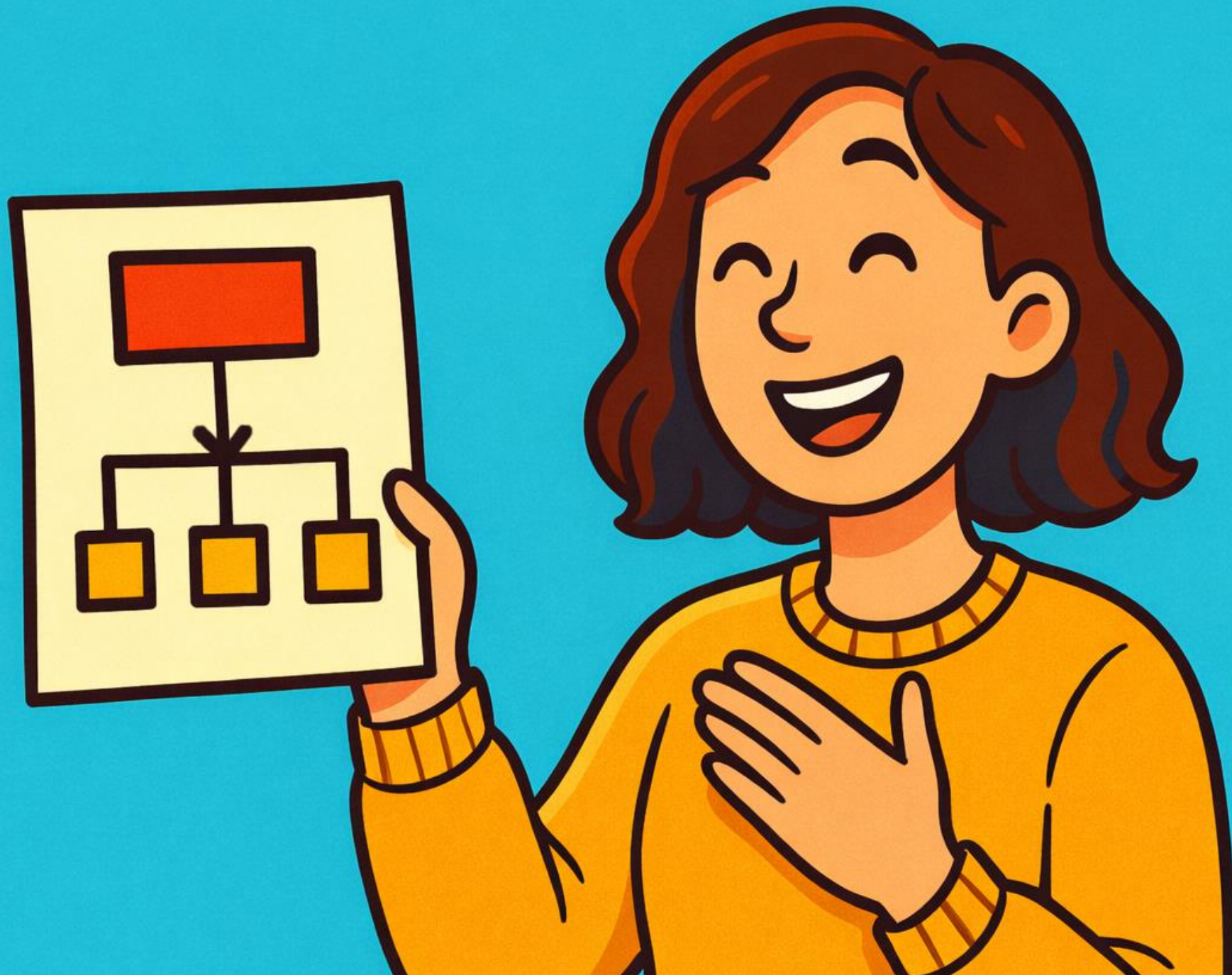
- [Communication Patterns with Jacqui Read](#)
- [OpenValue courses](#)
 - IT development for non-IT people
 - DevOps for Managers
 - Effective Communication for Software Developers
- Accompany users when they are using your application

INTERFACE



```
interface  
  Foo  
{  
  bar();  
}
```

Diagrams



Using Diagrams to See the Big Picture

- Existing diagrams may not be enough
- Use the “right” diagram
 - should help to understand a specific concept
- Use a legend, so everyone knows how to read the diagram

Recommendation



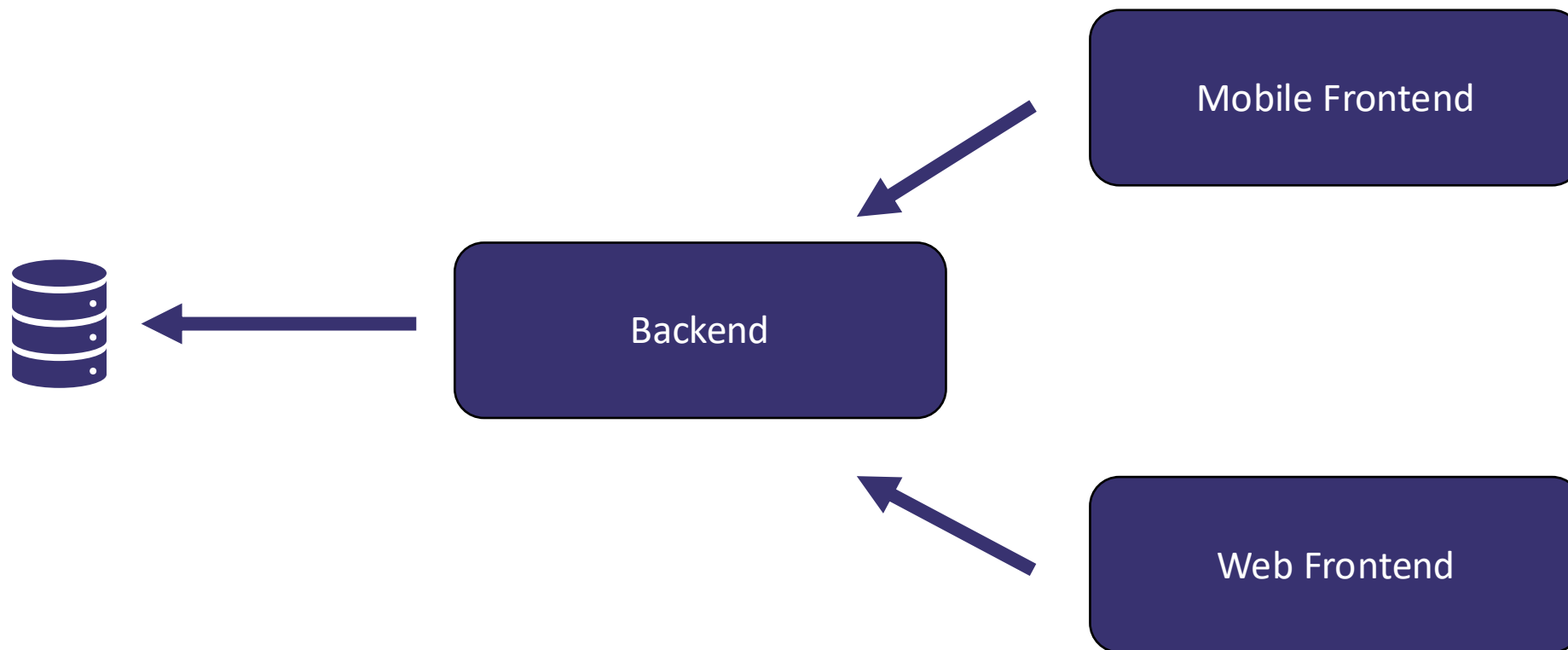
Jacqui Read - Mastering Visual Communication for Software Development - NewCrafts 2024

OPENVALUE
Better software, faster

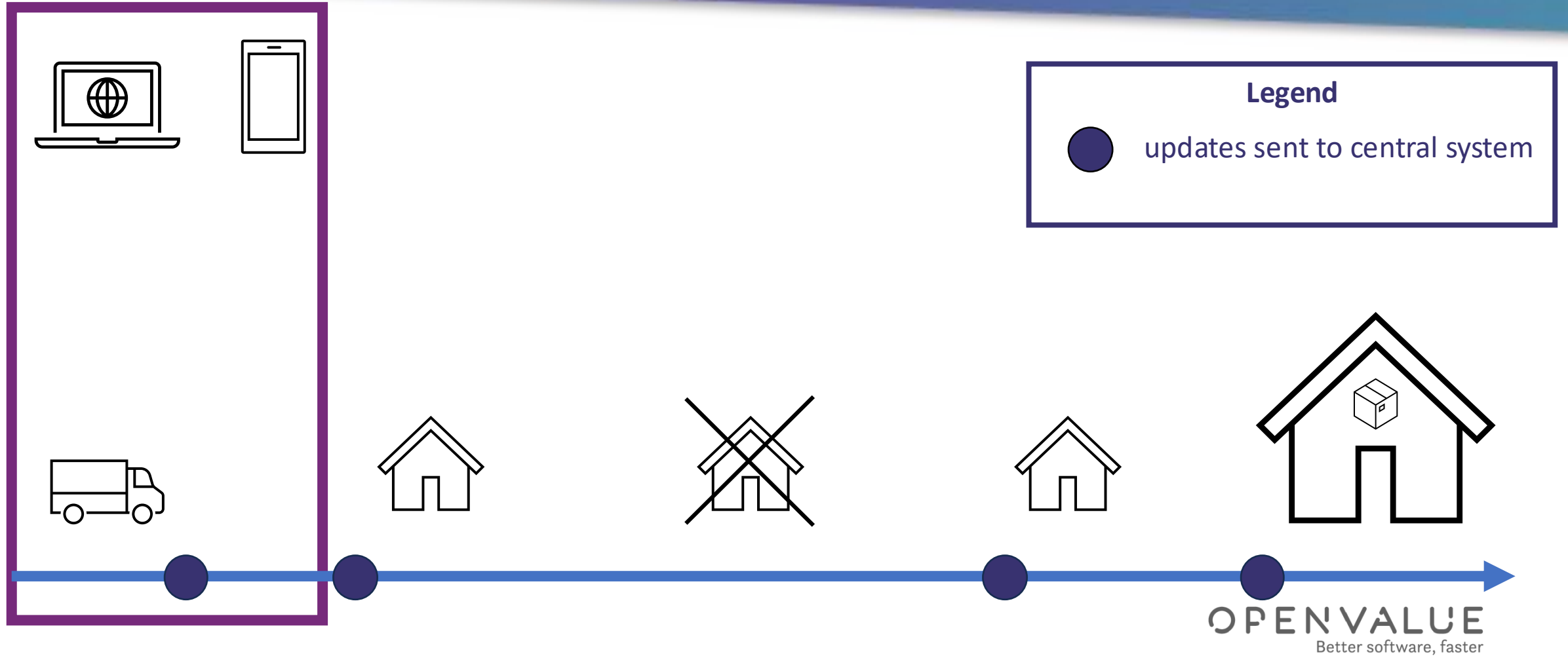
Mastering Visual Communication for Software Development (Jacqui Read)

- Different audiences need different levels of detail and different types of diagrams
- Consider color-blindness: at least check in greyscale
- High-level diagrams stay up to date for the longest time

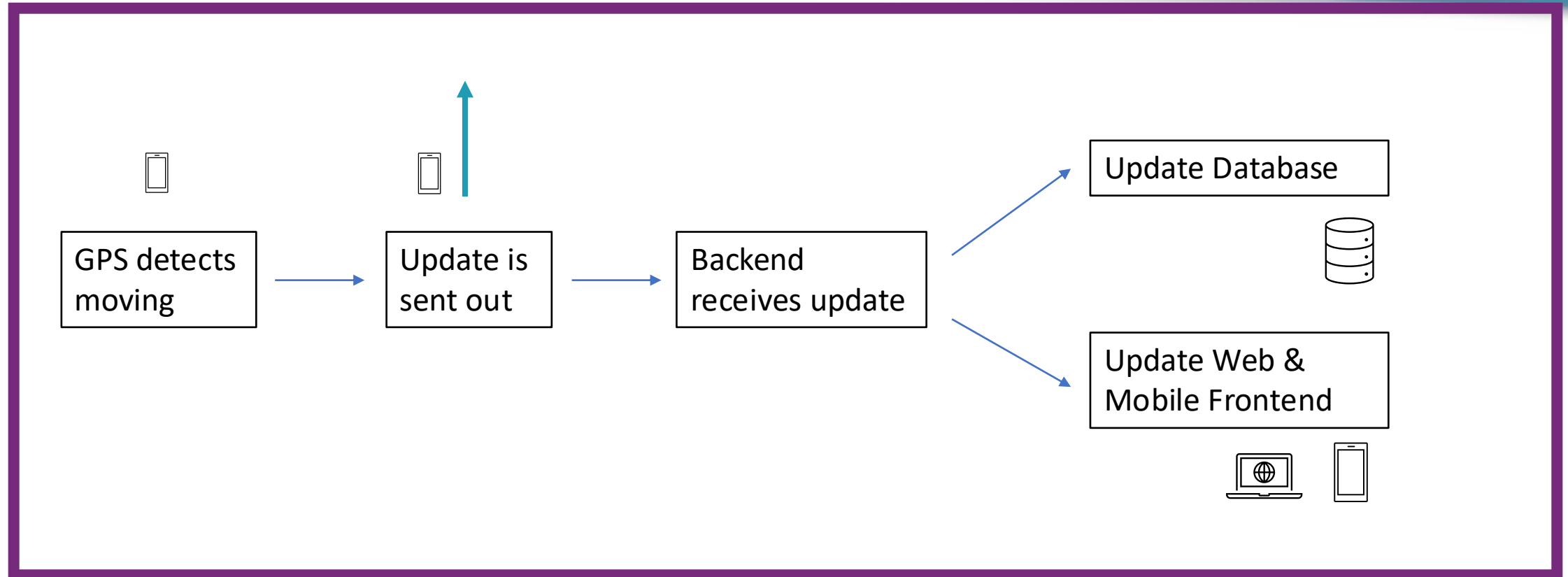
Big Picture



Intermediate Big Picture



Intermediate Big Picture



Organizational Scale and Knowledge Silos

Knowledge Fragmentation

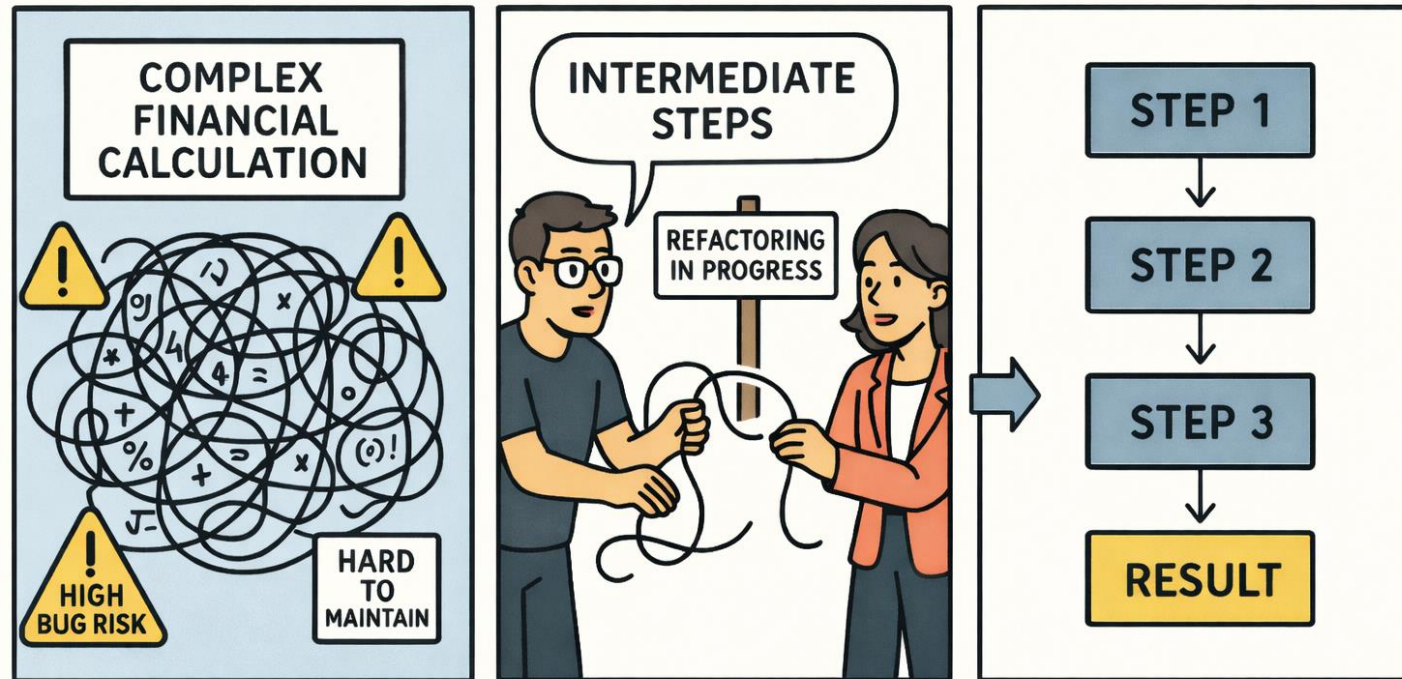
- Most developers do not know enough about the entire project
- Knowledge mostly only of one part of the project
- Combining different modules: one of the most difficult aspects
- Knowledge is not shared enough → collaborate more
 - Pair-Programming
 - [Bytesize Architecture Sessions](#)

Improving Documentation

- Diagrams & documentation should be understandable on its own
 - No one can remember everything
 - A big help when on-boarding employees
- Decisions should be documented: e.g. [decision logs](#)

Inherent Domain Complexity

Some Complexity Cannot Be Simplified



**YOU CAN'T ALWAYS REDUCE COMPLEXITY –
BUT YOU CAN MAKE IT MANAGEABLE.**

DDD (Domain-Driven Design)

Three-point summary of DDD

1. Focus on the core domain.
2. Explore models in a creative collaboration of domain practitioners and software practitioners.
3. Speak a ubiquitous language within an explicitly bounded context.

Source: [DDD Reference – Eric Evans](#)

Developer responsibility

- Static code analysis
- Code reviews
- Sticking to guidelines

Conclusion

- Code Quality is a shared responsibility
- Work on continuously improving the communication
 - Be aware of your assumptions
 - Use diagrams

Thank you for your attention



✉ annelore.egger@openvalue.ch

🐙 [anneloredev](https://github.com/anneloredev)

in [/in/anneloredev](https://www.linkedin.com/company/anneloredev)

✂ [@AnneloreDev](https://twitter.com/AnneloreDev)

▶ [@anneloredev](https://www.youtube.com/channel/UC...)

Thanks for your Feedback 😊



Why Bad Code Quality Is Not (Only) Your Fault

Further Resources

- [Devoxx Greece 2024 - Knowledge Management for the Technically Inclined by Jacqui Read](#)
- [Wardley Maps](#)
- [Summary of Eric Evans Talk](#)
["What I have learned about DDD since the book"](#)
- [Domain-Driven Design Europe](#) (and their [YouTube-Channel](#))
- [Agent skill: visual-explainer](#)